

Lời giải trận thứ tư HDU Multi-University 2021

Nguồn gốc: HDU Multi-University 2021 MINIEYE Cup, Contest 4 Solutions

contest/hdu-2021-minieye-04-solutions.pdf

1 1001 Calculus

Chú ý rằng mọi hàm được cho trong đề đều phân kỳ. Vì vậy chỉ cần kiểm tra xem tất cả hệ số của những hàm thành phần có bằng 0 hay không.

2 1002 Kanade Loves Maze Designing

Do đề cho phép thuật toán $O(n^2)$, ta có thể dùng kỹ thuật cộng trừ 1 để đếm số màu khác nhau.

Ký hiệu c_i là số lần màu i đang xuất hiện, và r là đáp án thu được khi DFS từ một điểm xuất phát đến đỉnh hiện tại. Khi đi vào một đỉnh, tăng c_i ứng với màu của đỉnh đó. Nếu đây là lần đầu màu này xuất hiện, tăng r thêm 1. Ghi lại đáp án tại đỉnh hiện tại. Khi rời đỉnh, giảm c_i ; nếu sau khi giảm màu này không còn xuất hiện nữa thì giảm r .

Độ phức tạp thời gian là $O(n^2)$, bộ nhớ $O(n)$.

Từ khóa: DFS.

3 1003 Cycle Binary

Với một chuỗi S , nếu biểu diễn được nó dưới dạng $kP + P'$ như trong đề, ta gọi P là một chu kỳ tuần hoàn của S , và độ dài của P là một chu kỳ của S .

Đề yêu cầu xét mọi chuỗi nhị phân độ dài n và gom đóng góp theo chu kỳ nhỏ nhất. Gọi $f(x)$ là số chuỗi nhị phân độ dài x có chu kỳ nhỏ nhất bằng x . Khi đó đáp án $g(n)$ là

$$g(n) = \sum_{x=1}^n \left\lfloor \frac{n}{x} \right\rfloor f(x).$$

Trước hết xét cách tính $f(x)$. Có 2^x chuỗi nhị phân độ dài x . Những chuỗi có chu kỳ $y < x$ và $y \mid x$ chắc chắn không được tính vào $f(x)$, vì chúng có chu kỳ nhỏ hơn. Ta muốn chứng minh rằng chỉ cần loại các chu kỳ như vậy là đủ.

Dùng định lý chu kỳ: nếu một chuỗi S có hai chu kỳ x, y và

$$x + y \leq |S| + \gcd(x, y),$$

thì $\gcd(x, y)$ cũng là một chu kỳ của S .

Xét chuỗi nhị phân T độ dài x và chuỗi S độ dài n được tạo từ các đoạn lặp của T . Giả sử T không có chu kỳ y thỏa $y < x$ và $y \mid x$, nhưng S lại có chu kỳ $y < x$. Nếu

$$x + y \leq n + \gcd(x, y),$$

thì theo định lý chu kỳ, $\gcd(x, y)$ cũng là chu kỳ của S . Đồng thời $\gcd(x, y) \mid x$, nên nó cũng là chu kỳ của T , mâu thuẫn với giả thiết. Trong trường hợp này ta có công thức

$$f(x) = 2^x - \sum_{\substack{y < x \\ y \mid x}} f(y).$$

Điều kiện trên tương đương với $x \leq \lfloor n/2 \rfloor + 1$. Với các $x > \lfloor n/2 \rfloor$, hệ số của $f(x)$ trong công thức của $g(n)$ đều bằng 1, nên không cần tính riêng từng giá trị đó. Ta biến đổi:

$$\begin{aligned} g(n) &= \sum_{x=1}^{\lfloor n/2 \rfloor} \left\lfloor \frac{n}{x} \right\rfloor f(x) + 2^n - \sum_{x=1}^{\lfloor n/2 \rfloor} f(x) \\ &= 2^n + \sum_{x=1}^{\lfloor n/2 \rfloor} \left(\left\lfloor \frac{n}{x} \right\rfloor - 1 \right) f(x). \end{aligned}$$

Từ công thức tính f cũng có

$$2^x = \sum_{y \mid x} f(y) = (f * 1)(x).$$

Dùng sàng Dujiao cho công thức tích chập trên, và dùng chia khối số học cho công thức của $g(n)$. Các giá trị tiền tố tại khoảng $2\sqrt{n}$ đầu mút do sàng Dujiao trả ra chính là những giá trị chia khối cần dùng. Sàng Dujiao cần tiền xử lý $f(x)$ với $x \leq n^{2/3}$; ta tính chúng bằng cách liệt kê bội số tương tự sàng Eratosthenes.

Độ phức tạp thời gian là

$$O\left(n^{2/3} \log n^{2/3} + \sum_T n^{2/3}\right),$$

bộ nhớ $O(n^{2/3})$.

4 1004 Display Substring

Nhị phân đáp án, sau đó dùng một cấu trúc dữ liệu hậu tố bất kỳ để kiểm tra.

Với mảng hậu tố, mọi tiền tố của mọi hậu tố chính là mọi chuỗi con. Duyệt mảng hậu tố; với mỗi hậu tố, tiền tố càng dài thì chi phí càng lớn, nên có thể nhị phân số tiền tố có chi phí không vượt quá giá trị đang kiểm tra. Sau đó trừ phần bị đếm lặp; phần lặp của mỗi hậu tố chính là giá trị tương ứng trong mảng height.

Với automaton hậu tố hoặc cây hậu tố, các chuỗi con phân biệt được ghi trên các đỉnh. Mỗi đỉnh biểu diễn những chuỗi dạng $\text{Suffix}(T, i) + S$ trong automaton hậu tố; trên cây hậu tố là chuỗi tương ứng sau khi đảo lại. Hậu tố của T càng dài thì chi phí chuỗi con càng lớn, nên lại có thể nhị phân độ dài để đếm số chuỗi con thỏa điều kiện. Mỗi lần kiểm tra thực hiện một phép nhị phân cho mỗi đỉnh.

Độ phức tạp thời gian là $O(n \log n \log k)$.

5 1005 Didn't I Say to Make My Abilities Average in the Next Life?!

Xét trước truy vấn trên đoạn $[1, n]$. Giá trị cần tính là

$$\left\lfloor \sum_{1 \leq l \leq r \leq n} \frac{\max a_{l..r} + \min a_{l..r}}{2} \middle/ \frac{n(n+1)}{2} \right\rfloor \pmod{10^9 + 7}.$$

Đây là bài kinh điển dùng cây Descartes hoặc ngăn xếp đơn điệu. Tách riêng đóng góp của giá trị lớn nhất và nhỏ nhất, rồi xử lý hai lần bằng cùng một kỹ thuật trong $O(n)$.

Cụ thể, với mỗi vị trí i , tìm đoạn lớn nhất $[p, q]$ thỏa $p \leq i \leq q$ và $\max a_p \dots q = a_i$. Khi đó đóng góp của a_i với vai trò giá trị lớn nhất là

$$a_i(i - p + 1)(q - i + 1).$$

Khi có nhiều truy vấn, có thể xử lý offline bằng ngăn xếp đơn điệu. Sắp xếp truy vấn theo đầu phải, rồi quét đầu phải từ nhỏ đến lớn. Với mỗi đầu trái, duy trì đáp án của truy vấn khi đầu phải là điểm đang quét.

Trong quá trình đưa phần tử mới vào ngăn xếp, ta có thể phải xóa một số điểm. Thao tác xóa một điểm khỏi ngăn xếp tương đương với cộng một cấp số cộng vào đáp án của các vị trí từ điểm đó đến điểm kế tiếp trong ngăn xếp, rồi gán giá trị lớn nhất mới. Có thể duy trì quá trình này bằng cây đoạn hỗ trợ cộng đoạn và gán đoạn. Sau khi xóa xong, với mỗi đầu trái, phần đóng góp của truy vấn phụ thuộc vào điểm có giá trị bên phải lớn nhất; phần này cũng có thể duy trì bằng cây đoạn.

Độ phức tạp thời gian là $O((n + m) \log n)$, bộ nhớ $O(n + m)$.

Ngoài ra còn có cách tính đóng góp của cực đại hoặc cực tiểu bằng hai lượt từ trái sang phải và từ phải sang trái để tìm phạm vi lớn nhất, rồi cộng theo công thức $a_i(i - p + 1)(q - i + 1)$. Cách này đạt $O(n \log n + m)$.

Từ khóa: offline, cây đoạn.

6 1006 Directed Minimum Spanning Tree

Giả sử đồ thị đầu vào liên thông mạnh. Giai đoạn cơ chu trình trong thuật toán Edmonds như sau.

1. Với mỗi đỉnh, chọn cung đi vào nó có trọng số nhỏ nhất.
2. Gọi C là một chu trình bất kỳ tạo bởi các cung đã chọn. Với mỗi cung (u, v) thuộc C , trừ $w(u, v)$ khỏi trọng số của mọi cung đi vào v .
3. Co C thành một đỉnh c và xóa các khuyên. Nếu đồ thị chỉ còn một đỉnh thì dừng.

Dựa trên quá trình này, xây một cây T . Khi đang co chu trình C , nếu cung (u, v) thuộc C , nối trong T một cung từ c đến v với trọng số $w(u, v)$. Có thể chứng minh rằng trọng số của DMST có gốc tại r bằng tổng trọng số của mọi lần co chu trình, trừ đi tổng trọng số các cung trên đường đi từ gốc của T đến lá r .

Với một bộ dữ liệu, tổng độ phức tạp là $O(m \log n)$.

7 1007 Increasing Subsequence

Định nghĩa $dp[i]$ là số dãy con tăng cực đại trong $a[1 \dots i]$ kết thúc tại $a[i]$. Giá trị $dp[i]$ đóng góp cho $dp[j]$ khi và chỉ khi giữa i và j không có phần tử nào có giá trị nằm giữa $a[i]$ và $a[j]$. Khởi tạo $dp[i] = 1$ khi và chỉ khi trước $a[i]$ không có phần tử nhỏ hơn $a[i]$. $dp[i]$ đóng góp vào đáp án khi và chỉ khi sau $a[i]$ không có phần tử lớn hơn $a[i]$. Cách trực tiếp cho độ phức tạp $O(n^2)$.

Dùng chia để trị và duy trì ngăn xếp đơn điệu cho hai nửa để giảm xuống $O(n \log^2 n)$.

Khi xử lý đoạn $[l, r]$, gọi m là trung điểm. Xét mọi phần tử trong $a[l \dots r]$ theo thứ tự giá trị tăng dần. Với $[l, m]$ và $[m + 1, r]$, xây hai ngăn xếp đơn điệu. Ở nửa trái, giá trị phần tử trong ngăn xếp tăng dần và vị trí giảm dần. Ở nửa phải, giá trị tăng dần và vị trí tăng dần.

Khi xử lý một phần tử ở nửa trái, chỉ cần đưa nó vào ngăn xếp. Khi xử lý phần tử $a[i]$ ở nửa phải, dùng ngăn xếp để tìm phần tử $a[j]$ nhỏ hơn nó và nằm gần bên phải nhất trong $[m + 1, i - 1]$. Sau đó tìm trong ngăn xếp bên trái phần tử đầu tiên lớn hơn $a[j]$ và phần tử cuối cùng nhỏ hơn $a[i]$. Đóng góp của đoạn $[l, m]$ vào $dp[i]$ là tổng các giá trị dp của những phần tử nằm giữa hai vị trí này trong ngăn xếp.

8 1008 Lawn of the Dead

Tính tổng số ô rồi trừ đi số ô không thể tới, ta thu được số ô có thể tới.

Theo đề bài, ô có mình không thể tới. Vì đối tượng chỉ di chuyển sang phải và xuống dưới, một ô trở thành không thể tới nếu cả ô bên trái lẫn ô phía trên của nó đều không thể tới. Khi đó ô này tiếp tục ảnh hưởng đến các ô bên phải và bên dưới nó.

Không gian rất lớn nhưng số mình hữu hạn. Ta xử lý từng hàng từ trên xuống, và trong mỗi hàng sắp xếp các mình. Với mỗi mình, tìm đoạn liên tiếp không thể tới từ trái sang phải bắt đầu ở ô góc trên phải của nó, tức $(x - 1, y + 1)$. Khi đó đoạn tương ứng trên hàng x cũng không thể tới.

Dùng cây đoạn để truy vấn đoạn và phủ đoạn. Sau khi xử lý xong một hàng, hỏi số ô không thể tới trên hàng đó, cộng dồn, rồi lấy tổng số ô trừ đi kết quả này.

9 1009 License Plate Recognition

Chiếu biển số theo phương dọc xuống một chiều: cột nào có ký tự # thì ghi 1, cột còn lại ghi 0. Sau đó tìm biên trái và phải của từng ký tự.

Cần chú ý đặc trưng của ảnh: mọi chữ số và chữ cái tiếng Anh đều có hình chiếu là một đoạn liên tiếp, nhưng ký tự chữ tượng hình không nhất thiết như vậy. Vì thế ta tìm các đoạn của chữ số và chữ cái từ phải sang trái trước; phần còn lại là ký tự đầu biển số.

10 1010 Pony Running

Đây là bài DP xác suất có chu trình, dùng cách kinh điển là khử Gauss.

Với mỗi ô, viết phương trình tuyến tính

$$dp(i, j) = p_{ij0}dp(i - 1, j) + p_{ij1}dp(i + 1, j) + p_{ij2}dp(i, j - 1) + p_{ij3}dp(i, j + 1) + 1,$$

còn với ô ngoài bảng thì $dp(i, j) = 0$. Như vậy có nm phương trình tuyến tính. Gọi vector giá trị DP là x , ma trận hệ số là A , vector hằng là B ; cần giải

$$Ax = B.$$

Trước hết dùng khử Gauss để tìm A^{-1} , tốn $O((nm)^3)$.

Truy vấn yêu cầu tổng các phần tử của x , tức $x = A^{-1}B$. Với thao tác sửa, ta duy trì A^{-1} bằng đồng nhất thức Woodbury:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}.$$

Đổi bốn xác suất của một ô tương đương với đổi bốn giá trị trong ma trận, nên có thể tách thành bốn lần sửa hạng một trên ma trận A .

Ví dụ, nếu cộng Δ vào hàng i , cột j của A , ta có

$$A^{-1}U = A^{-1}(:, i), \quad VA^{-1} = A^{-1}(j, :),$$

và

$$(C^{-1} + VA^{-1}U)^{-1} = \left(\frac{1}{\Delta} + A^{-1}(j, i) \right)^{-1} = \frac{1}{1/\Delta + A^{-1}(j, i)}.$$

Do đó mỗi lần sửa tốn $O((nm)^2)$.

Mỗi truy vấn tính tổng kỳ vọng số bước để ra khỏi bảng của mọi ô bằng một lần nhân ma trận để tìm $x = A^{-1}B$, cũng tốn $O((nm)^2)$. Tổng độ phức tạp là

$$O((nm)^3 + q(nm)^2).$$

Vì đồ thị là lưới, cũng có thể dùng khử trực tiếp hoặc phương pháp chọn trụ để thông qua bài này.

11 1011 Travel on Tree

Đề bài yêu cầu, với nhiều truy vấn, tính tổng trọng số các cạnh trong cây con được tạo bởi đường đi giữa mọi cặp đỉnh của một đoạn.

Khó gộp hai đoạn trực tiếp, nên dùng thuật toán Mo. Bài toán chuyển thành: khi thêm hoặc xóa một đỉnh khỏi cây con hiện tại, tính đóng góp của đỉnh đó vào tổng trọng số.

Gọi đoạn hiện tại là $[l, r]$, S là tập các đỉnh từ l đến r , và T' là cây con tạo bởi mọi đường đi giữa các cặp đỉnh trong S . Giả sử cần thêm đỉnh u . Cố định gốc của cây ban đầu, và gọi v là LCA của mọi đỉnh trong S . Khi thêm u , có hai trường hợp.

1. Nếu $\text{lca}(u, v) = v$, cần tìm đỉnh w đầu tiên xuất hiện trong T' trên đường từ u lên gốc. Đóng góp là

$$\text{dep}(u) - \text{dep}(w).$$

2. Nếu $w = \text{lca}(u, v) \neq v$, đóng góp là

$$\text{dep}(u) + \text{dep}(v) - 2 \text{dep}(w).$$

Trong trường hợp thứ nhất, cần tìm nhanh w . Dựa vào thứ tự DFS, nếu sắp xếp các đỉnh trong S theo thứ tự DFS, chỉ cần tìm tiền nhiệm x của u trong S và hậu nhiệm y của u trong S . Khi đó w là đỉnh sâu hơn trong hai đỉnh $\text{lca}(u, x)$ và $\text{lca}(u, y)$. Cần xử lý riêng khi x hoặc y không tồn tại.

Nếu duy trì S bằng cây đoạn hoặc cây cân bằng, độ phức tạp sẽ là $O(n\sqrt{n} \log n)$, vẫn chưa đạt yêu cầu. Phân tích thao tác xóa cho thấy nếu duy trì các đỉnh trong S bằng danh sách liên kết, ta có thể tính đóng góp khi xóa trong $O(1)$ và cũng hỗ trợ hoàn tác trong $O(1)$. Vì vậy chỉ thực hiện xóa và hoàn tác, tức dùng Mo có rollback. Truy vấn LCA dùng RMQ-LCA với thời gian $O(1)$.

Tổng độ phức tạp là $O(n\sqrt{n})$. Lời giải chuẩn dùng danh sách liên kết tự cài và một ngăn xếp ghi lại tổng trọng số cần rollback. Nếu dùng `std::list`, hoặc khi rollback lại tính lại đóng góp, hằng số có thể tăng đáng kể.